CITS3003 Project Set Up Guide

Follow the instructions below to build and run the project.

Project Directory

Download the starting code for the labs (`cits3003_project.zip`) and unzip it. The `cits3003_project` directory contains `CMakeLists.txt`, `lib`, `res`, `lab1` and other files. You can build and run the project on Linux, MacOS, or Windows, similar to how you did for the labs.

If you have not previously built and run any CITS3003 labs, please read the following guidelines carefully. Otherwise, you should already be familiar with the process.

Getting ready to build (Linux)

On Linux, you'll need to install a few dependencies before building the project. For Debian-based distributions (like Ubuntu or PopOS), run the following command to get everything you need:

```
sudo apt update && sudo apt install git cmake build-essential
libx11-dev libxrandr-dev libxinerama-dev libxcursor-dev libxi
-dev xorg-dev
```

If something is missing, ask for help and let us know so we can update this list.

Getting ready to build (MacOS)

You'll need a build chain (gcc/g++, clang, etc.), which is usually provided by Xcode. If you don't have one, install the Xcode command line tools or Xcode itself. If compilation fails, try updating Xcode.

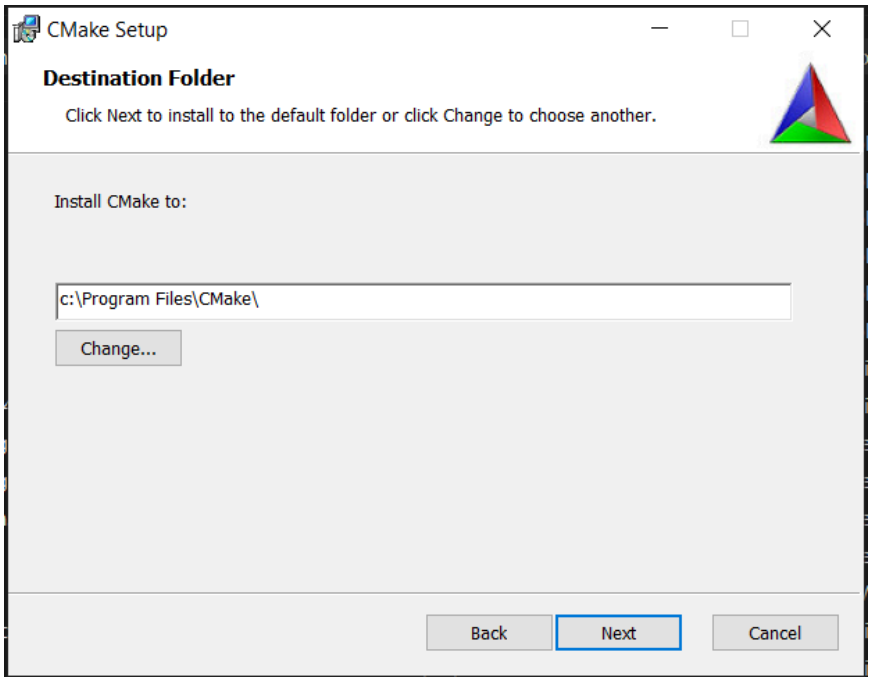
You'll also need CMake. Get it from Homebrew or https://cmake.org/download/. Make sure it's added to your PATH so you can use it from the command line. To do

this, run the CMake application, go to the Tools menu, select "How to Install for Command Line Use," and follow one of the presented options.

Getting ready to build (Windows)

Install the following:

1. **MinGW-w64 toolchain**: Follow steps 1–7 in the guide in https://code.visualstudio.com/docs/cpp/config-mingw#_installing-the-mingww64-toolchain.
2. **CMake**: Download the .msi Windows x64 Installer from https://cmake.org/download. Run the .msi file to install CMake. During installation, you can choose where to install CMake (e.g., `C:\Program Files\CMake`). Remember this location since you'll need it to add CMake to your PATH environment variable.
   After installation, add its bin directory to your Windows PATH environment variable. (See step 7 in the guide above for instructions on adding environment variables. Typically, you just need to append `bin` to the installation location, e.g., `C:\Program Files\CMake\bin`).



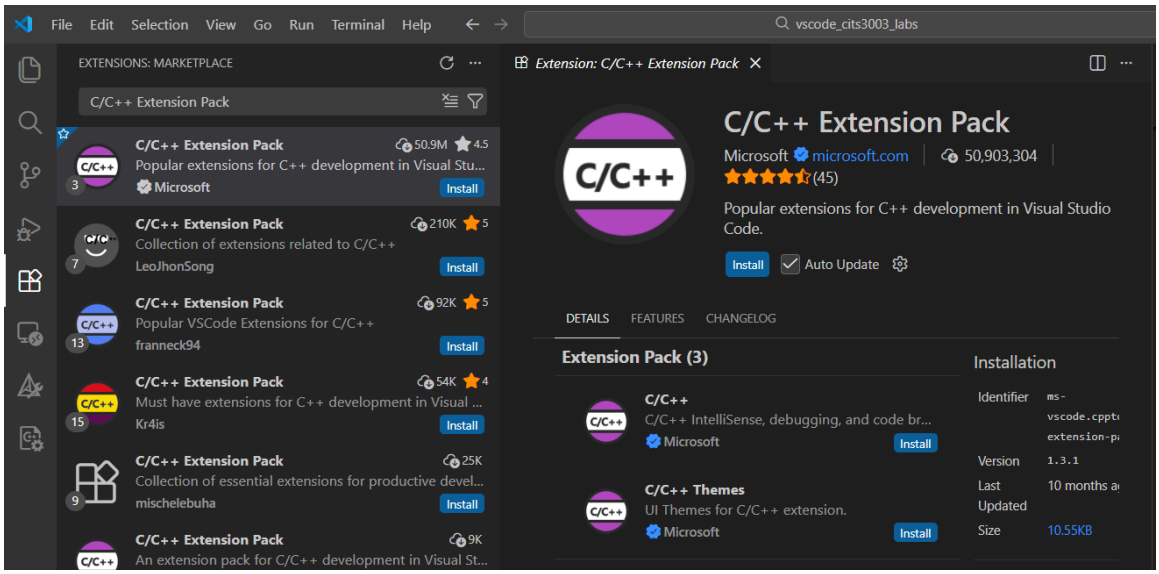
3. **Ninja**: Download the prebuilt Ninja binary (`ninja-win.zip`) from https://github.com/ninja-build/ninja/releases. Extract the .zip file and add the `ninja-win` directory to your PATH environment variable.

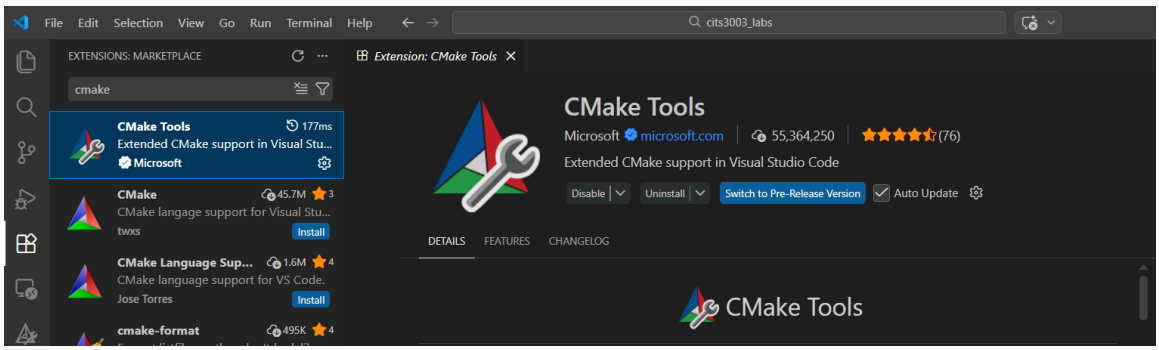
Building and Running

You can choose to build and run your codes using VS Code or CLion. The details for each option are given below. **The tutorial below explains how to build and run the lab code. The same steps can be extended and applied to your project.**

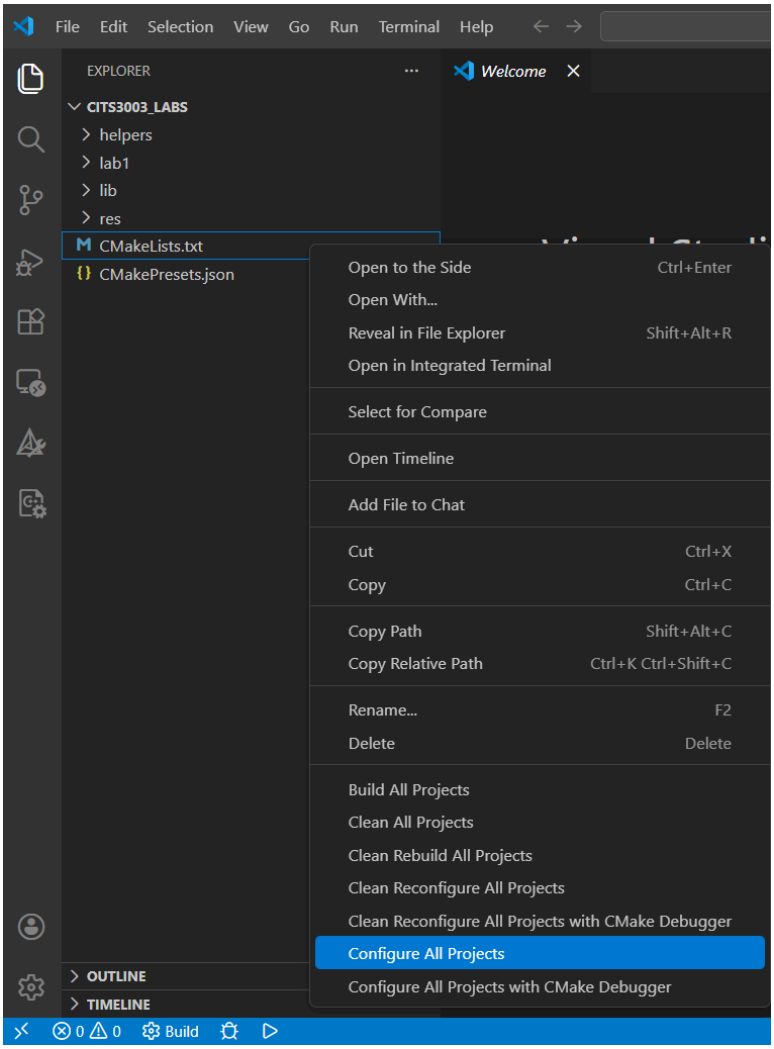
VS Code

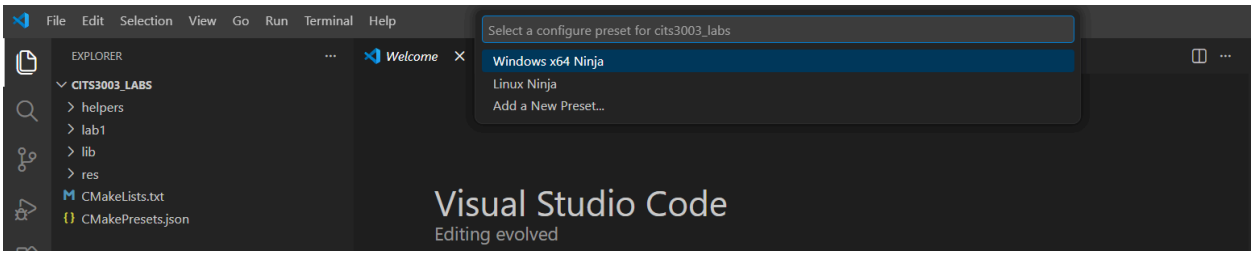
1. Download and install VS Code.
2. Install the `C/C++ Extension Pack` and `CMake Tools` extensions for VS Code.



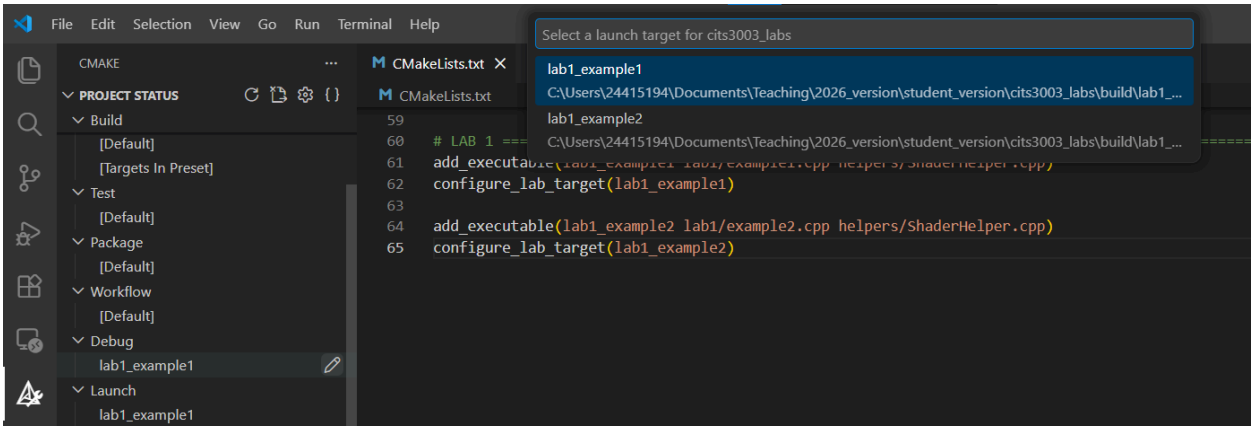


3. Open `cits3003_labs` folder in VS Code, trusting the folder if prompted.
4. Select the `Windows x64 ninja` option when the configure preset options appear. If the options don't show up, right-click on `CMakeLists.txt` and choose `Configure All Projects` to display the list.

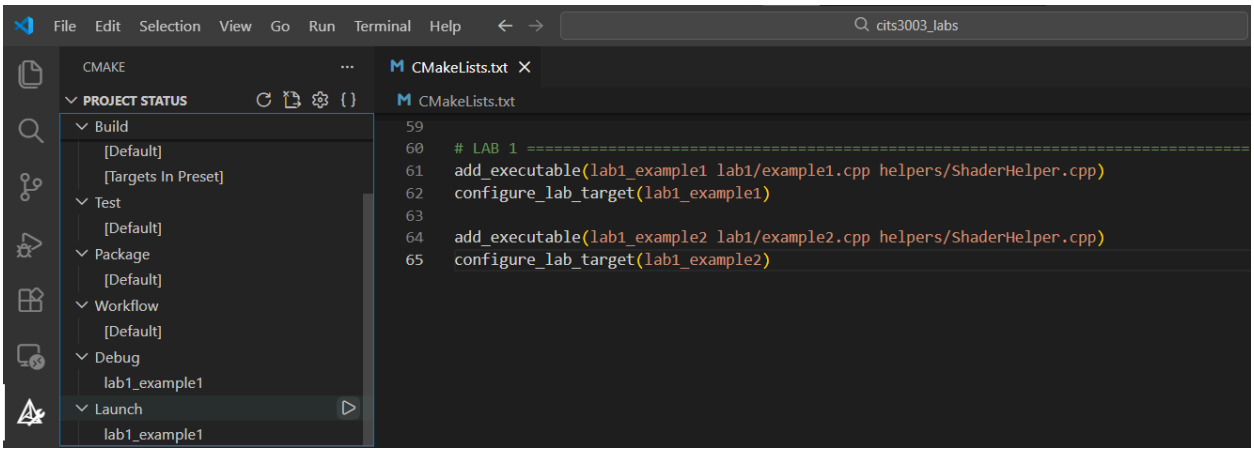




5. On the CMake menu on the left, you'll find Debug and Launch profiles for debugging and release builds. Click the pencil icon next to each profile to select the build target.



6. To build and debug, click the play button next to the Debug profile. To build and run normally, click the play button next to the Launch profile.



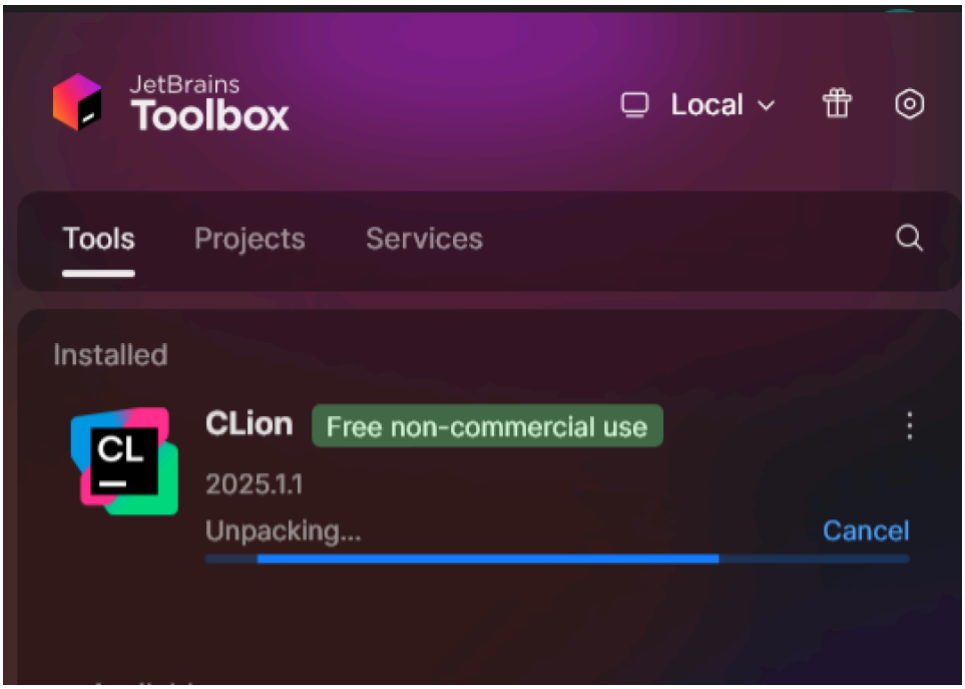
Whenever you add a new `.cpp` file for a lab, you'll need to add them to `CMakeLists.txt` so you can build and run it. For example, if you write a new `lab1/q1circle.cpp` file, you need to add two lines to the end of `CMakeLists.txt`:

```
add_executable(lab1_q1circle lab1/q1circle.cpp helpers/Shader
Helper.cpp)
```

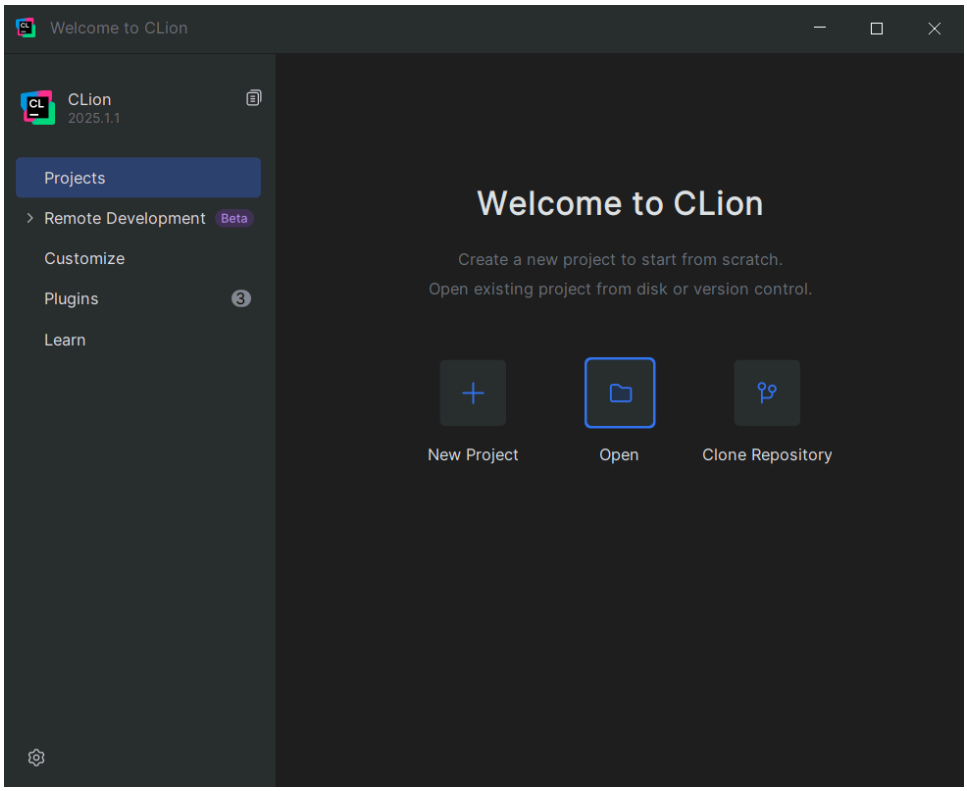
```
configure_lab_target(lab1_q1circle)
```

CLion

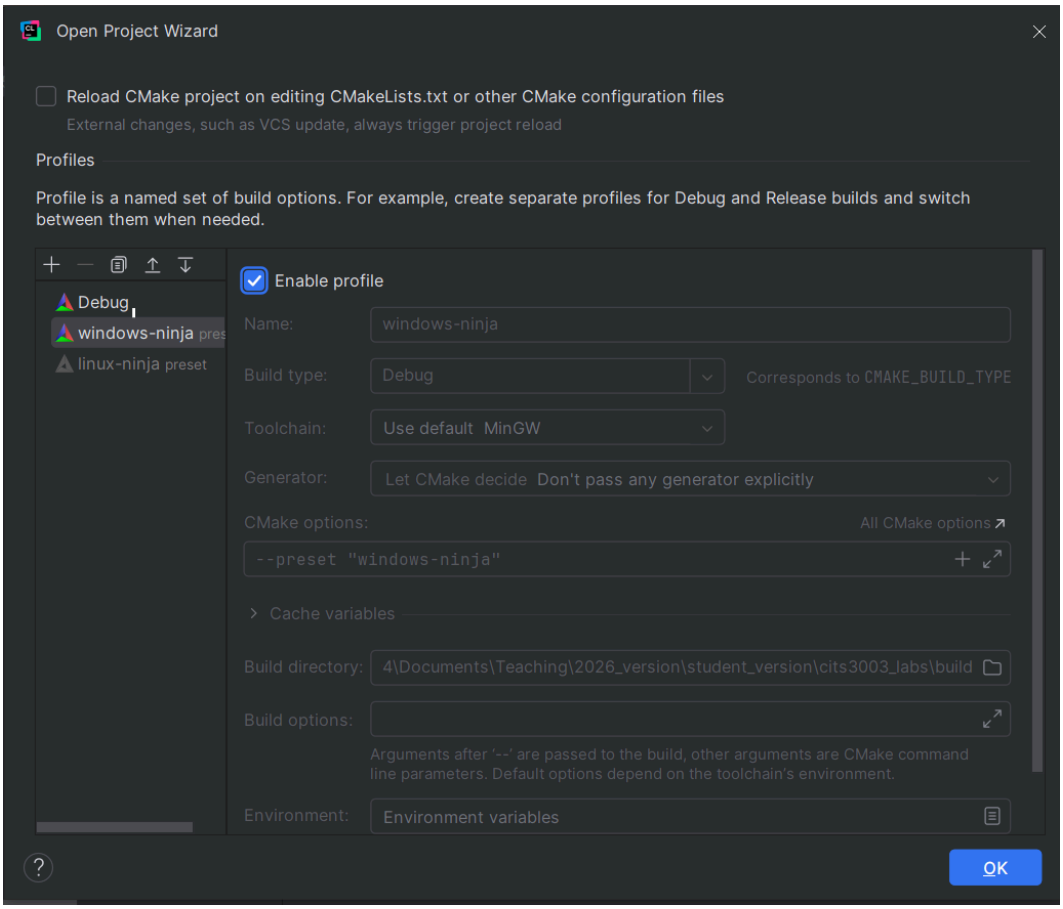
1. Download Jetbrain Toolbox.
2. Open Jetbrain Toolbox, search for CLion, and install it.



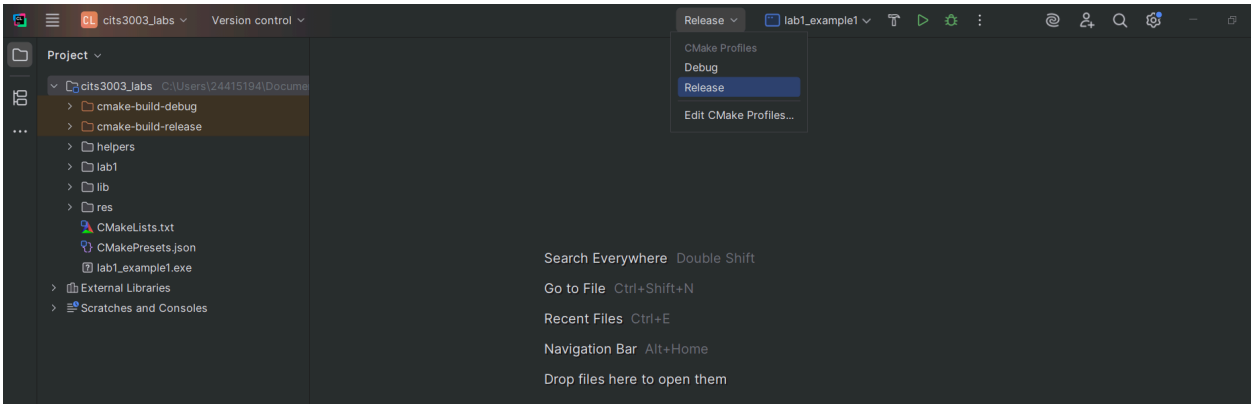
3. Register/log in. You can use your student email to get a free license.
4. Open `cits3003_labs` folder in CLion. It may then ask you if you trust this project, say yes.



5. It then may ask you to setup your CMake profiles, I suggest clicking the + at the top of the list on the left to automatically add a Release profile for compiling with optimizations enabled. You can also click on `windows-ninja` preset profile and choose `Enable profile`. Then click OK to continue.

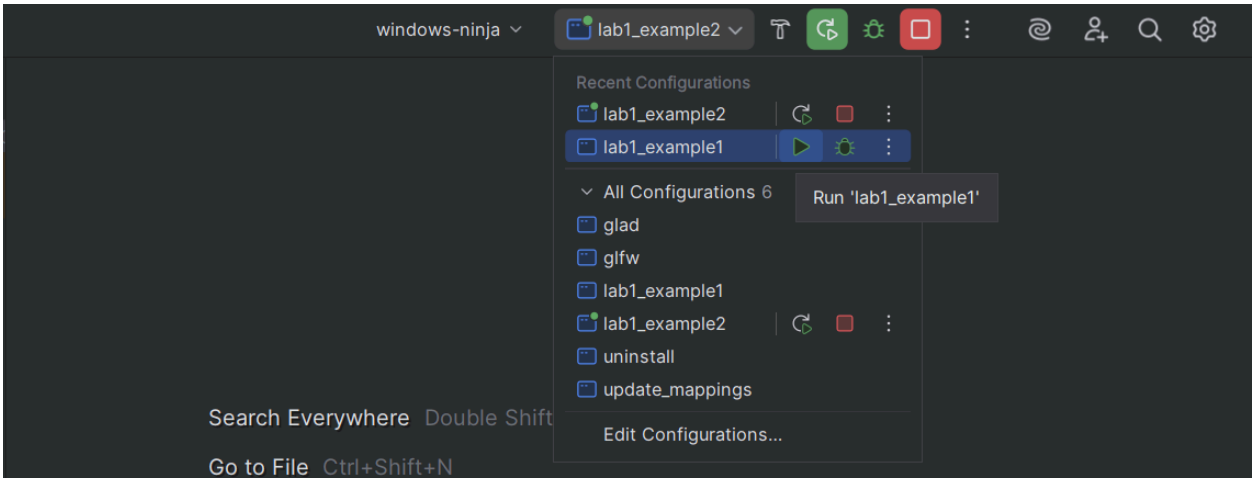


6. At the top, you can switch between Debug/Release profiles and choose the build target. There are three icons for build, run, and debug. Note that debugging will only work properly if you have the Debug profile selected.



7. To run multiple targets at once (e.g., for side-by-side comparison), open the **Configuration Switcher** dropdown.

Instead of switching, click the Run (Green Play) icon directly next to any target in the list. This launches that specific file as a separate process while keeping your current selection active.



Whenever you add a new `.cpp` file for a lab/project, you'll need to add them to `CMakeLists.txt` so you can build and run it. For example, if you write a new `lab1/q1circle.cpp` file, you need to add two lines to the end of `CMakeLists.txt`:

```
add_executable(lab1_q1circle lab1/q1circle.cpp)
configure_lab_target(lab1_q1circle)
```

Building via cmake and executing in Linux

Open a terminal window. Type in the terminal window:

```
cd cits3003_project
```

The command above would change the working directory to the specified directory name. In the terminal window, type: `ls`. You should see other directories such as `helpers`, `lab1`, and `res`, as well as CMakeLists.txt.

Then to generate build files, run (these only needs to be re-run if you update CMakeLists.txt

```
cmake -S . -B cmake-build-debug -DCMAKE_BUILD_TYPE=Debug
cmake -S . -B cmake-build-release -DCMAKE_BUILD_TYPE=Release
```

Then to build the debug profile, run:

```
cmake --build cmake-build-debug
```

Then to build the release profile, run:

```
 cmake --build cmake-build-release
```

Then to run the last built profile, run:

```
 ./cits3003_project
```